

Regulatory Change Monitor

From a new Fedlex publication to a lawyer-approved, department-specific client alert: how we classify Swiss regulatory changes, match them to client companies, and keep a lawyer in control of every word.

3 + 1

sources: Fedlex SPARQL (new AS publications, upcoming entry into force, open consultations) + organisers' dataset

81 %

of update × company pairs decided by transparent rules without an LLM call (full live-Fedlex run: 144 of 178)

0

alerts reach a client without a named LEXR lawyer's approval; enforced in the backend, not the UI

1 The problem and our answer

2 System overview

3 Ingest: sources and deduplication

4 Classification

5 Matching

6 Worked examples

7 Drafting, routing and citations

8 Lawyer review and responsible AI

9 Evaluation

10 Limitations and next steps

Stack: FastAPI · SQLAlchemy/SQLite · React + Vite · LLM: Swisscom **Apertus 1.5 70B** (Swiss-hosted), OpenAI as alternative, and a deterministic *mock* mode that runs the whole product with no network and no keys.
This tool assists lawyers. It does not give legal advice.

1 · The problem and our answer

Swiss federal law changes every week: new acts and ordinances in the Official Compilation (AS), changes that come into force on a fixed date, and consultations that close on a deadline. A law firm like LEXR has to answer the same question over and over, for dozens of clients: **does this change matter to this company, who in the company has to act, and by when?**

Doing this by hand does not scale. Letting an LLM send alerts on its own is not acceptable for legal content. We built a pipeline in between. The machine does the reading, sorting and first draft. A lawyer approves, edits or rejects every alert before a client sees it. Every decision the machine takes comes with a reason a lawyer can check.

Precise

Classification happens once per update and does not depend on any client. Matching combines weighted rules with an LLM judgment, so the result is specific to each company.

Explainable

Every match stores its rule hits, score, the LLM's reason, the model version and the prompt version. Non-matches are stored too, with the reason they were dropped.

Controlled

No alert without lawyer approval. Drafts without citations cannot be approved. Every step is audit-logged with actor, model and source version.

2 · System overview

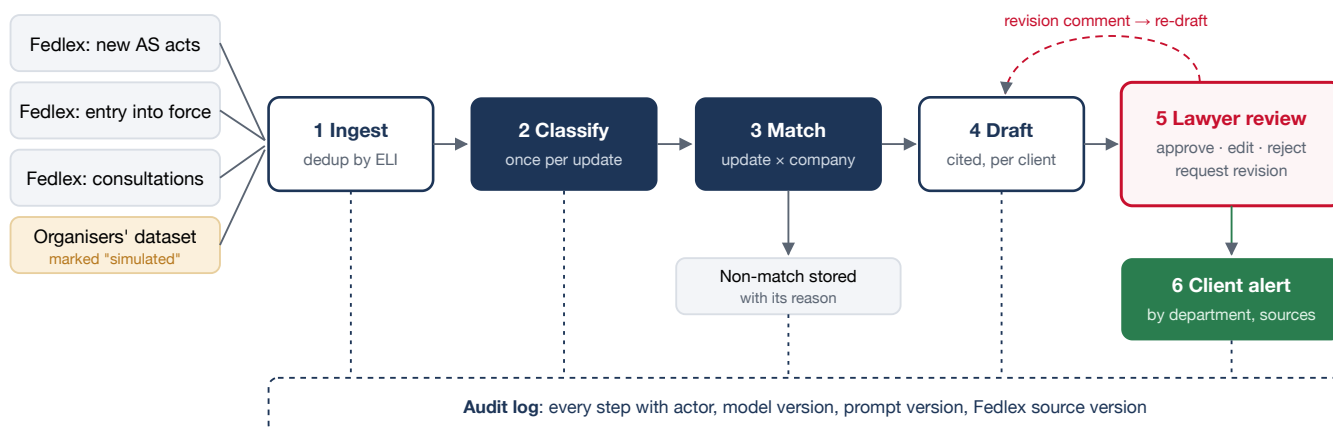


Figure 1. The pipeline. Steps 2 and 3 (dark) are the core of this document. The AI steps are pure functions (dict in, dict out). All database writes and audit entries happen in a separate service layer.

Client onboarding: what the matcher knows about a company

Each client fills in a structured profile. These fields are what both the rules and the LLM compare against:

Field	Example (Nimbus AI AG)
Industry, description, size band, HQ canton	B2B SaaS / AI · Zurich AI start-up selling document analytics · 11–50 · ZH
Jurisdictions	CH, EU
Activity flags (10 fixed keys)	processes_personal_data, customer_profiling, offers_ai_features, public_sector_clients
Legal context, key challenges	"Subject to the revised FADP ... 40 % of customers in DE/FR, so the EU AI Act applies ..."
Topics of interest	AI governance, data protection, cybersecurity, accessibility

3 · Ingest: sources and deduplication

We query the public **Fedlex SPARQL endpoint** (no key needed) for the three kinds of change named in the challenge:

1. **New publications** in the Official Compilation (AS, ELI prefix `/eli/oc/`) since a date.
2. **Upcoming entry into force**: acts whose `jolux:dateEntryInForce` lies in the next 180 days.
3. **Open consultations** (`jolux:Consultation`) whose end date has not passed.

For each act we fetch the HTML text and split it into articles, so drafts can later cite *Art. 7* instead of "the law".

Deduplication. The same legal change often shows up twice, for example as a new AS publication and again as an upcoming entry into force. The key is the **ELI URI**. A second sighting adds a source entry to the existing update and fills gaps such as the entry-into-force date, but it never creates a second alert.

Demo safety. With `DEMO_MODE=true` the ingest reads a cached Fedlex snapshot. The "Ingest live" button still calls the live endpoint. The organisers' `xlsx` rows are loaded as updates flagged `is_simulated`, and the UI always marks them as simulated.

4 · Classification

Goal: describe each update *once*, in a fixed vocabulary, without looking at any client. This keeps the cost per update constant however many clients LEXR has. It also gives the matcher structured facts to compare instead of free text.

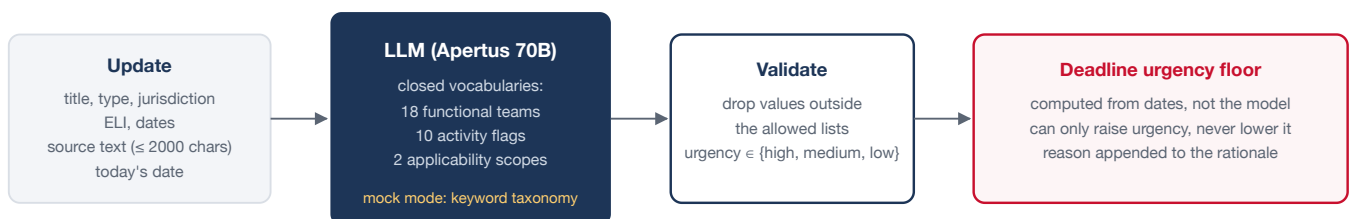


Figure 2. Classification. The model proposes, code constrains: anything outside the allowed vocabularies is dropped, and hard deadlines override the model's sense of urgency.

Output (stored on the update, versioned)

Field	Meaning	Used by
topics	Short English topic tags	Matching (overlap with client's topics)
functional_teams	Which teams would need to act (closed list of 18)	Routing to the client's departments
triggered_flags	Kinds of company the update imposes obligations on (closed list of 10)	Matching (flag hits, sector veto)
applies_to	all_legal_entities or employers: obligations that bind every company of a kind, whatever its sector	Matching (scope hits, overrides the sector veto)
affected_business_types	Free-text descriptions ("payment providers")	Matching (phrase found in the profile)
urgency, rationale	high / medium / low plus 1–2 sentences on why	Review queue sorting, drafts
model_version, prompt_version	e.g. apertus:swiss-ai/Apertus-v1.5-70B, classify-v3	Audit log, reviewer screen

The classification prompt (shortened)

All prompts share one guardrail block. The full prompts are in `backend/app/ai/prompts.py`. Any change to a prompt bumps its version constant.

SYSTEM: SHARED GUARDRAILS

```
You assist LEXR, a Swiss law firm. You are NOT giving legal advice. A LEXR lawyer reviews everything you write before a client sees it.
- Use only the source text and metadata provided. Do not invent facts, dates, articles or obligations.
- If the source does not say something, say it is unclear rather than guessing.
- Answer with ONE JSON object only.
Task: classify a Swiss/EU regulatory update independently of any specific client.
```

USER: CLASSIFY-V3

```
Today's date: {today}
Regulatory update: {title, type, jurisdiction, ELI, dates, source_text_excerpt}
Allowed functional_teams: [Legal, Compliance, Product, Engineering, Data, Security, ... 18]
Allowed triggered_flags (the kinds of COMPANY this update imposes obligations on):
- finma_supervised: banks, insurers, fintechs, payment providers ... (incl. everyone under the GwG)
- public_sector_clients: companies that SELL to public bodies. Do NOT use it just because the text mentions authorities. ... 10 flags
Only choose a flag if companies with that activity must check or change something.
Allowed applies_to (obligations that bind every company of a kind, regardless of sector):
- all_legal_entities · - employers
Use applies_to even when the law is motivated by one sector: e.g. a beneficial-owner register introduced for AML reasons still binds every legal entity -> finma_supervised AND all_legal_entities.
Urgency: consider how soon the change applies relative to today's date.
Return JSON: {"topics", "functional_teams", "triggered_flags", "affected_business_types", "applies_to", "urgency", "rationale"}
```

Why applies_to exists (the fix that went into classify-v3). In an earlier version, the Federal Act on the Transparency of Legal Entities (TJPG, AS 2026/323) was classified as `finma_supervised` only, because it is part of the anti-money-laundering package. The matcher then vetoed it for every client that is not a financial intermediary. But the TJPG obliges every Swiss legal entity to identify its beneficial owners. Separating "which sector motivated this" (flags) from "who is bound by it" (scopes) fixed this whole class of false negatives. See the worked example in section 6.

Deadline-based urgency floor

LLMs are unreliable with date arithmetic. So urgency gets a minimum value computed from the update's dates. The model can make an update more urgent than the floor, but never less.

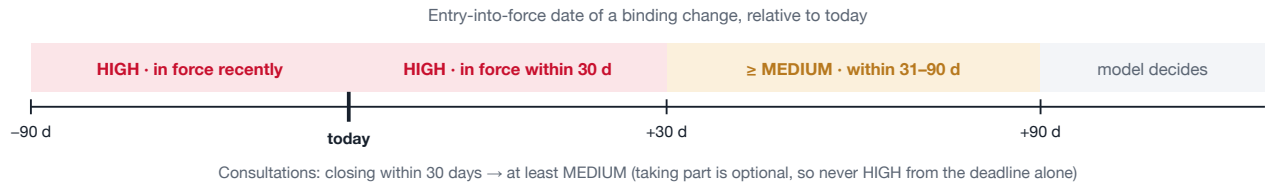


Figure 3. Urgency floor (`rules.apply_deadline_floor`). Example: TJPG enters into force on 2026-10-01. Classified on 2026-09-23 → "Urgency raised from medium to high: enters into force in 8 days."

Mock mode: the same interface without a model

With `LLM_PROVIDER=mock` the classifier is a keyword taxonomy: 14 topics, each with DE/FR/IT/EN regexes, the activity flags it implies and the teams that own it (e.g. *data protection*: `datenschutz|personendaten|protection des données|...` → flags `processes_personal_data`, `customer_profiling` → teams Legal, Data, Security). Applicability scopes are only derived from the *title*. An anti-money-laundering act that mentions beneficial owners in its body does not bind every company, but the act on the transparency of legal entities does. This mode is our demo safety net, and it is the baseline in section 9.

5 • Matching

Goal: for every (update, company) pair, decide whether the company plausibly has to check or change something, and say why. We use two stages. **Transparent rules come first:** they are cheap and deterministic, and a lawyer can read them. **The LLM comes second**, only for candidates the rules could not rule out.

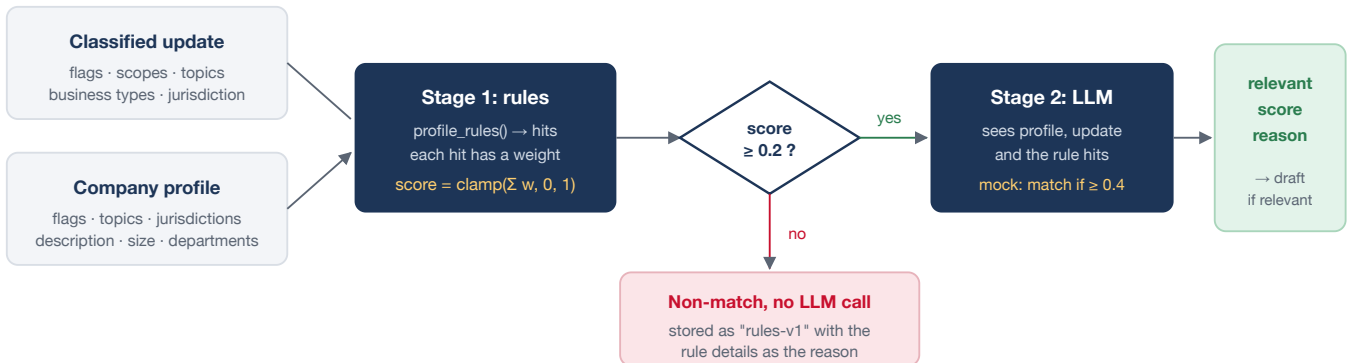


Figure 4. Two-stage matching (`ai/matching.py`). Clear non-matches never cost a model call. Each rule hit gets a weight, the weights are summed into a score, and the rule hits are passed to the LLM as "pre-screening, may be incomplete or wrong".

Stage 1: the rules and their weights

Rule	Fires when ...	Weight
<code>flag:<flag></code>	the update triggers an activity flag the company has declared (e.g. <code>offers_ai_features</code>)	+0.5
<code>flag:processes_personal_data</code>	same, for the generic flag that almost every client has, so it counts as weak evidence	+0.2
<code>scope:all_legal_entities</code> <code>scope:employers</code>	the update binds every company or every employer, and the company is one (employer = more than 10 staff, an HR department, or monitors employees). Only for Swiss law or a jurisdiction the company is active in.	+0.4
<code>topic:<topic></code>	a classified topic overlaps a topic of interest from the profile (substring either way), once per topic	+0.4
<code>business_type:<type></code>	an affected business type ("payment provider") appears in the company's industry, description, legal context or challenges	+0.3
<code>sector:mismatch</code>	the update triggers a <i>sector</i> flag (FINMA, health, nonprofit, public sector, minors) the company does not have, and no scope applies. This is a veto.	-1.0
<code>sector:general-scope</code>	same, but a scope applies: the sector parts are irrelevant, the general obligation remains. The veto is lifted.	0.0
<code>jurisdiction:EU-not-active</code>	EU update, company not active in the EU	-0.4

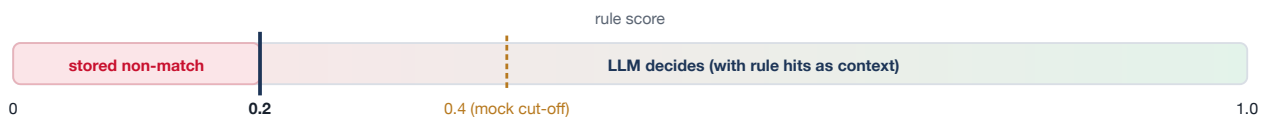


Figure 5. Thresholds. A single scope hit (0.4) is enough to reach the LLM on its own, and in mock mode it is enough to match. A sector veto (-1.0) outweighs anything short of a scope.

Stage 2: the matching prompt (shortened)

SYSTEM: GUARDRAILS + MATCH TASK

[shared guardrails]

Task: decide whether a regulatory update is relevant to one specific client company.

Be conservative: relevant means the company plausibly has to check or change something.

If a rule hit says the update applies to every legal entity or every employer and the company is one, treat it as relevant unless the text clearly excludes this company.

USER: MATCH-V2

```
Client profile: {name, industry, description, size_band, hq_canton, jurisdictions,
                active_flags, legal_context, key_challenges, topics, department_names}
Regulatory update: {metadata + source_text_excerpt (≤ 1200 chars)}
Deterministic rule hits (pre-screening, may be incomplete or wrong):
[{"rule": "scope:all_legal_entities", "detail": "...", "weight": 0.4}, ...]
Return JSON: {"relevant": true|false, "relevance_score": 0..1,
              "reason": "2-3 sentences, specific to this company, referring to profile fields"}
```

Why rules first, then an LLM?

- **Cost and speed.** In our full live-Fedlex run, 144 of 178 pairs (81 %) were settled by rules alone. Most federal publications (customs rates for maize seedlings, heritage inventories) concern none of the clients.
- **Explainability.** A rule hit is a sentence a lawyer can check against the profile: "Update concerns *finma_supervised*, which the company declared."
- **Nuance where it pays off.** Rules cannot read the act. The LLM can tell that an AML amendment which widens the definition of "advisors" still concerns a payment provider.
- **The rule hits frame the model.** We pass them as evidence, not as the answer, so the model can overrule a weak hit, and has to give a reason when it does.

Safety properties of the matcher

- **Non-matches are first-class records.** A lawyer can see *why* an update was not sent to a client, which helps when auditing missed alerts.
- **Approved work is frozen.** Re-running the pipeline never re-matches or overwrites a pair whose draft a lawyer has approved.
- **Everything is versioned.** Each match stores `model_version` (`rules-v1`, `mock:heuristic-v0` or `apertus:...`) and `prompt_version`, and each one is written to the audit log.
- **Parallel but rate-limited.** LLM calls run in a thread pool with a global request-rate cap (Apertus: 5 req/s) and a response cache keyed by prompt, so re-runs with unchanged prompts are almost instant.

6 · Worked examples

Two demo clients: **Nimbus AI AG** (Zurich AI SaaS, CH + EU, no financial licence) and **Helvetia Pay SA** (Geneva payment provider, FINMA-supervised, CH only). Rule hits are the current code (`classify-v3` / `match-v2`). LLM reasons are from Apertus 1.5 70B, shortened.

A · Federal Act on the Transparency of Legal Entities (TJPG), AS 2026/323			in force 2026-10-01
Classification: topics <i>financial market regulation, corporate transparency</i> · flags <i>finma_supervised</i> · applies_to <i>all_legal_entities</i> · urgency HIGH (raised by the deadline floor: in force in 8 days)			
Company	Rule hits	Score	Outcome
Helvetia Pay	flag:finma_supervised +0.5 · scope:all_legal_entities +0.4 · topic:financial market regulation +0.4 · topic:corporate transparency +0.4 · ...	1.00	MATCH "...FINMA-supervised payment provider that must identify beneficial owners under AMLA. The TJPG mandates enhanced identification ... directly impacts its AML compliance."
Nimbus AI match-v1	sector:mismatch -1.0	0.00	NO MATCH no LLM call. Wrong: every AG must keep a beneficial-owner record.
Nimbus AI match-v2	scope:all_legal_entities +0.4 ("every client is a company") · sector:general-scope 0.0 ("sector-specific parts do not apply, but the update also applies generally")	0.40	MATCH the candidate goes to the LLM, which is told to treat general-scope hits as relevant unless the text excludes the company.

B · Guidance for providers of high-risk AI systems (dataset R001, EU, simulated)			SIMULATED
Classification: flags <i>offers_ai_features</i> · topic <i>AI governance</i> · jurisdiction EU · urgency HIGH			
Company	Rule hits	Score	Outcome
Nimbus AI	flag:offers_ai_features +0.5 · topic:AI governance +0.4	0.85*	MATCH "...offers AI-powered features and lists AI governance as a key topic; the update clarifies obligations for high-risk AI providers, relevant for its EU customers."
Helvetia Pay	flag:offers_ai_features +0.5 · jurisdiction:EU-not-active -0.4	0.10	NO MATCH no LLM call: "EU update, but company is not active in the EU."

* LLM relevance score. The rule score was 0.9.

7 · Drafting, routing and citations

For each match, the drafter writes a client-specific alert in the client's preferred language. It gets the profile, the client's departments (with IDs), the **match reason**, and the source **split into articles** in the original language.

Department routing connects classification and onboarding. The classified `functional_teams` are compared with the functions each department covers. If nothing matches, the alert goes to the department that covers Legal.

```
Task: draft a client alert. EVERY factual
statement
about the law must carry a citation (ELI URI +
article).
Statements you cannot cite must not be made.
Return JSON: {"summary" (≤ 120 words),
  "affected_departments": [{id, name, why}],
  "next_steps": [{action, department, due}],
  "urgency", "citations": [{claim, eli,
                           article, quote}]}
```

Example draft excerpt: Helvetia Pay × TJPG (German, as requested)

"Das neue Transparenzgesetz (TJPG) verpflichtet Helvetia Pay SA, die wirtschaftlich berechtigten Personen ... zu identifizieren, zu dokumentieren und innerhalb eines Monats ... an das Transparenzregister zu melden."

Departments: Compliance & AML ("must ensure identification and reporting under Art. 7–9 TJPG"), Risk & Data.

Citation: eli/oc/2026/323 · Art. 7

„Die Gesellschaft muss ihre wirtschaftlich berechtigten Personen identifizieren ..."

Urgency on the draft is floored by the same deadline rule as the classification, whatever the model says. In mock mode a template draft is built from the match data and clearly labelled as such.

8 · Lawyer review and responsible AI

The review screen

The lawyer sees the draft **next to the original source text**, with the rule hits, the LLM reason, the model and prompt version, and automatic quality warnings:

NO_CITATIONS	blocks approval (checked in the backend too)
CITATION_WITHOUT_SOURCE / _ARTICLE	citation lacks an ELI or article
SIMULATED_SOURCE	based on the organisers' dataset
TEMPLATE_DRAFT	generated without an LLM
NO_DEPARTMENTS	nobody routed

Actions: **approve** · **edit** (versioned, with a snapshot in the revision history) · **reject** · **request revision** (the model re-drafts using the lawyer's comment).

Guarantees built into the code

- One way to create an alert.** `Alert` rows are only created in `review._create_alert`, which is only called from `approve()` and refuses unless the draft is approved by a named lawyer.
- Approved means final.** An approved draft cannot be edited, re-matched or regenerated.
- Full audit trail.** Ingest, classify, match, draft, edit, approve, reject and deliver each write an audit entry (actor, model, prompt version, source version) in the same transaction.
- No evaluation leakage.** The jury columns of the dataset are read only by `scripts/evaluate.py`. They never reach rules, prompts or the DB.
- Data sovereignty.** Default model: Swisscom Apertus, hosted in Switzerland. Mock mode needs no external service at all.
- Not legal advice.** The prompts forbid legal conclusions. Every client alert shows "Reviewed by LEXR on ...", the source links and a disclaimer.

9 • Evaluation

The organisers' dataset contains eight simulated updates (R001–R008) with the jury's expected routing teams. We classify each one and compare our `functional_teams` with the jury's set (Jaccard similarity, precision, recall).

Classifier	Mean Jaccard	Precision	Recall	Reading
Keyword taxonomy (mock, <code>classify-v3</code>)	0.66	0.71	0.88	Tight and predictable; R006 (accessibility) matched exactly.
Apertus 1.5 70B (<code>classify-v3</code>)	0.40	0.42	0.83	Finds the right teams, but adds Compliance, Engineering and Security almost every time.

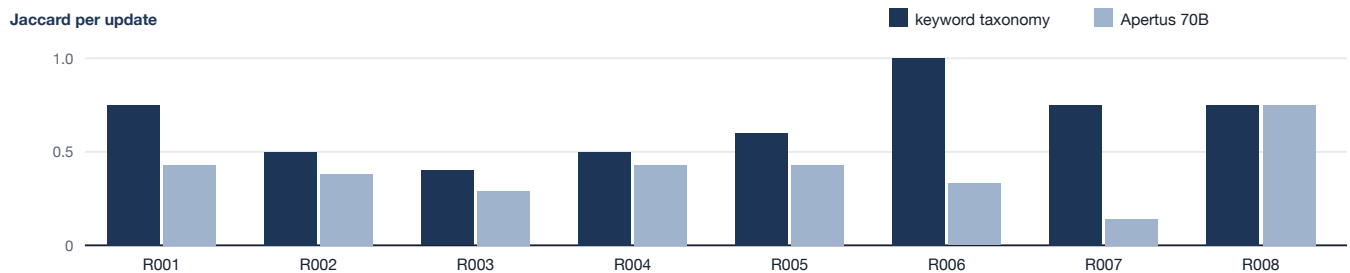


Figure 6. Routing agreement with the jury, per dataset row (`make eval`).

How we read this. Both approaches have high recall: the teams the jury expects are almost always included. The LLM's low precision means it over-routes: an alert reaches more departments than necessary. For legal monitoring, over-routing is the safer error (a lawyer removes a department with one edit, but a missed department is invisible). Still, it is the clearest next improvement: a few-shot prompt using the taxonomy's team mapping, or intersecting LLM teams with the taxonomy when both are available. The dataset has eight rows, so these numbers show a direction and should not be read as a benchmark.

10 • Limitations and next steps

Known limitations

- Citations are not yet checked mechanically. The next step is to verify that each cited article exists and each quote is a verbatim substring of the source.
- Deduplication uses only the ELI. A second, non-Fedlex source would need fuzzy matching (SR number, normalised title).
- Company scopes are coarse (legal entity, employer). Size thresholds (e.g. audit duties) and canton-specific law are not modelled yet.
- No authentication. The demo uses a role switcher, so the hosted demo should sit behind HTTP basic auth.
- The LLM over-routes teams (section 9).

Next steps

- A feedback loop: lawyer edits to departments and rejections become labelled data to tune rule weights and few-shot examples.
- Scheduled ingest (daily Fedlex delta) plus email delivery of approved alerts.
- More scopes (`listed_companies` , `vat_registered` , size bands) and canton rules.
- Consultations as an opportunity: suggest that clients comment on drafts that affect them before the deadline.
- Supertext integration for high-quality DE/FR/IT client versions.

Run it

```
make dev # mock mode, no keys, http://localhost:5173
docker compose up --build # one container, http://localhost:8010
LLM_PROVIDER=apertus + APERTUS_API_KEY in backend/.env → make reset # real model
```